DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SMART SUSTAINABLE CITY ENERGY INTELLIGENCE PLATFORM (SSEIP)

Course Code & Name: CSA16 – Data Warehousing and Data Mining

1. Problem Statement and Problem Formulation

The Chennai Energy and Sustainability Authority (CESA) requires an integrated analytical platform to process heterogeneous data collected from smart meters, solar systems, public buildings, EV charging stations, weather stations, traffic sensors, residential and commercial establishments, waste-management systems and citizen applications. The objective is to convert this data into decision-support information for reducing excessive energy consumption and waste, increasing renewable-energy adoption and improving sustainable infrastructure.

The solution is formulated as five connected analytical tasks:

Task	Analytical Question	DWDM Technique	Output
1	Which buildings/zones consume excessive energy?	OLAP analysis + clustering	High-consumption profiles and groups
2	Who may become high-energy consumers next cycle?	Classification	High-energy prediction
3	Which areas have similar sustainability characteristics?	K-Means clustering	Zone/building clusters
4	Which activities occur together?	Association rule mining	Support, confidence and lift rules
5	Where should interventions be prioritized?	Integrated decision analysis	Solar/EV/efficiency recommendations

2. Objectives and Expected Outcomes

- Design a data warehouse architecture and dimensional model for city energy intelligence.
- Prepare heterogeneous city data through cleaning, integration, transformation and reduction.
- Predict consumers/buildings likely to become high-energy consumers.
- Group similar energy and sustainability profiles using clustering.
- Discover co-occurring energy-consuming activities using association rules.
- Convert analytical findings into clearly separated business and sustainability recommendations aligned with SDG 7, SDG 11 and SDG 12.

Expected outcomes include a cleaned analytical dataset, dimensional warehouse design, prediction model, cluster profiles, association rules, visualizations and an intervention-priority framework.

3. Requirements, Constraints and Assumptions

Category	Details
Data	A suitably constructed sample dataset is used because no original CESA dataset is supplied.
Tools	Python 3, Pandas, NumPy, scikit-learn and Matplotlib.

Warehouse	Star schema with fact tables and conformed dimensions.
Prediction target	High-energy next cycle is represented as a binary class.
Clustering	Numeric variables are standardized before K-Means.
Rule mining	Transactions are derived from categorized energy-related activities.
Constraint	Recommendations are based on analytical patterns and are not treated as direct proof of causality.

This report follows the requirement that algorithm selection is justified, outputs are interpreted and data-mining results are distinguished from business recommendations.

4. Application of Relevant Course Knowledge / Concepts

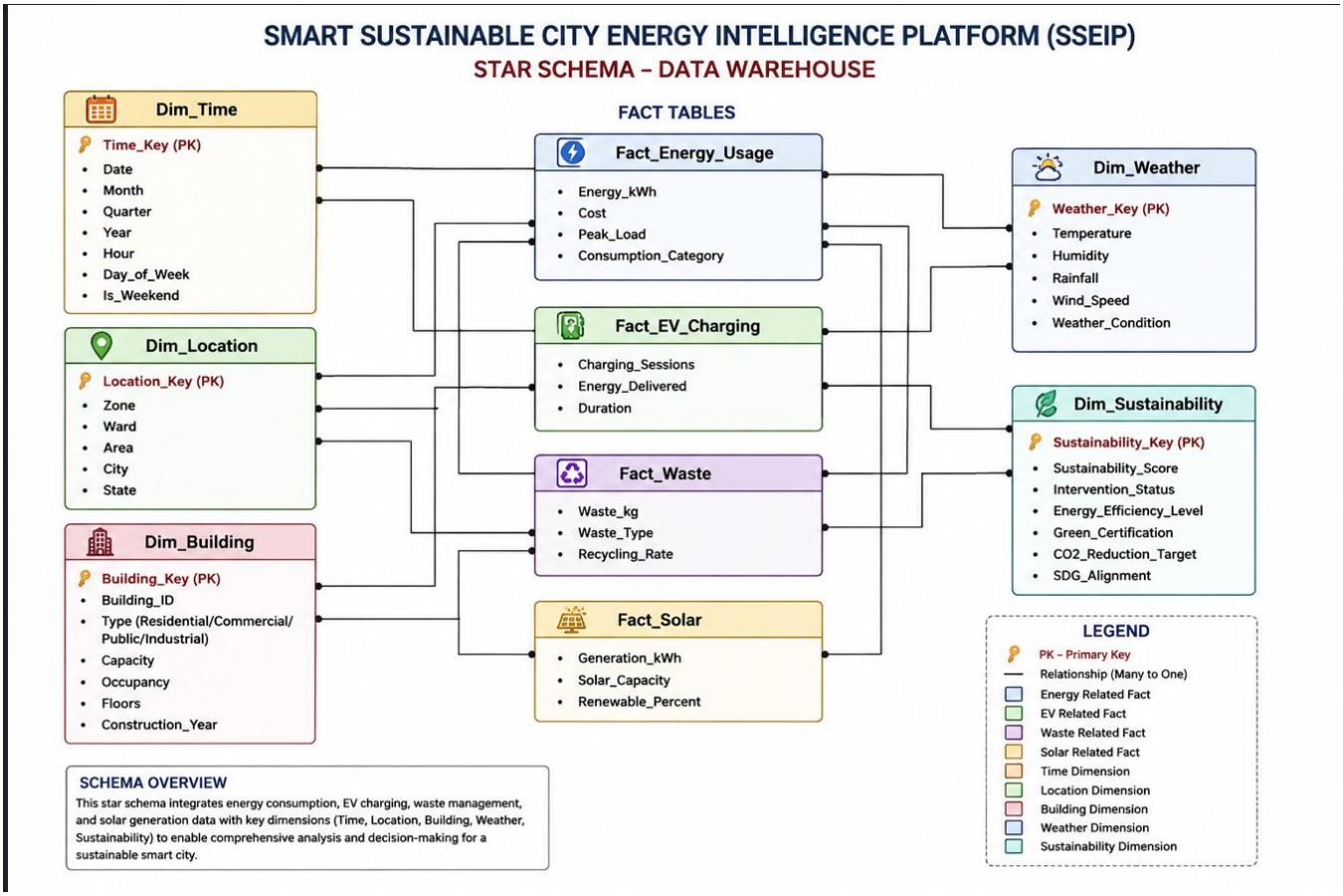
CO	Course Knowledge Applied
CO1	Data warehouse architecture, ETL, OLAP and dimensional modelling.
CO2	Data cleaning, integration, transformation, normalization and reduction.
CO3	Supervised classification, train/test evaluation, accuracy, precision, recall and F1-score.
CO4	K-Means clustering, silhouette-based evaluation and visualization.
CO5	Association rule mining using support, confidence and lift.

5. Data Warehouse Architecture and Dimensional Model

A layered architecture is proposed: Data Sources → Staging Area → ETL/ELT → Central Data Warehouse → Data Marts/OLAP → Data Mining and Decision Dashboard.

Data sources include smart meters, solar systems, EV charging stations, weather stations, traffic sensors and waste systems. ETL validates and integrates records. The warehouse stores historical data. Analytical models operate on prepared data marts.

5.1 Star Schema



Justification: the star schema is selected because decision-support queries frequently aggregate measures by time, zone and building. Conformed dimensions allow energy, solar, EV and waste facts to be analyzed together.

6. Data Preprocessing

Step	Method	Purpose
Cleaning	Remove duplicates and handle missing numeric values with median	Improves data quality
Cleaning	Validate non-negative energy, waste and solar values	Removes invalid records
Integration	Join records using common time/location/building identifiers	Creates a unified analytical view
Transformation	Create High_Energy_Next_Cycle and activity categories	Supports mining tasks
Normalization	StandardScaler for clustering	Prevents large-scale variables dominating distance
Reduction	Use selected analytical variables and remove irrelevant fields	Reduces dimensionality and noise

Sample dataset used in this implementation contains 500 records and variables representing zone, building type, temperature, occupancy, solar capacity, traffic, EV sessions, waste, energy consumption, renewable-energy percentage and sustainability score.

7. Proposed Methodology / Design

The methodology follows the sequence below:

- Collect or construct heterogeneous source data.
- Clean invalid, missing and duplicate values.
- Integrate data using common identifiers.
- Transform raw values into mining-ready features and categories.
- Load historical measures into the dimensional warehouse.
- Apply Random Forest classification for high-energy prediction.
- Apply K-Means clustering for similarity analysis.
- Apply association-rule mining for co-occurring activities.
- Evaluate models and interpret outputs.
- Translate validated findings into separate sustainability recommendations.

7.1 Algorithm Selection Justification

Technique	Reason for Selection
Random Forest	Handles nonlinear relationships and mixed operational factors; reduces overfitting through multiple trees and provides strong baseline performance.
K-Means	Efficient and suitable for grouping standardized numerical consumption and sustainability profiles.
Silhouette Score	Measures cluster separation and cohesion to support selection of a meaningful k value.
Association Rules	Directly identifies combinations of activities; support, confidence and lift provide understandable decision measures.

8. Algorithms / Pseudocode

8.1 Preprocessing

INPUT: Raw city energy datasets
1. Load datasets
2. Remove duplicate records
3. Handle missing values
4. Validate ranges and remove/repair invalid values
5. Integrate by time, location and building keys
6. Create derived attributes and categories

7. Standardize selected numerical features where required
OUTPUT: Clean analytical dataset

8.2 Random Forest Classification
INPUT: Prepared features X and target y
1. Split data into training and testing sets
2. Train Random Forest classifier on training data
3. Predict class labels for test data
4. Compute accuracy, precision, recall and F1-score
5. Select the model only after metric-based validation
OUTPUT: High-energy consumer predictions

8.3 K-Means Clustering
INPUT: Standardized clustering dataset
1. For each candidate k, fit K-Means
2. Compute silhouette score
3. Select the k with the best useful separation
4. Fit final K-Means model
5. Assign cluster labels
6. Summarize each cluster profile
OUTPUT: Similar energy/sustainability groups

8.4 Association Rules
INPUT: Activity transactions
1. Count item and item-pair occurrences
2. Compute support for each candidate rule
3. Compute confidence A → B
4. Compute lift = confidence / support(B)
5. Keep rules above chosen thresholds
6. Rank rules by usefulness
OUTPUT: Co-occurring activity patterns

9. Implementation / Source Code and Tools Used

```
# ============================================================

# SMART SUSTAINABLE CITY ENERGY INTELLIGENCE PLATFORM (SSEIP)

# Data Warehousing and Data Mining Assignment

#

# Algorithms Used:

# 1. Random Forest Classification

# 2. K-Means Clustering
```

```
# 3. Association Rule Mining
#
# Graphs:
# - Energy Consumption Distribution
# - Classification Metrics
# - Silhouette Score
# - Cluster Visualization
# - Confusion Matrix
# ============================================================


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    silhouette_score
)
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
# ============================================================
```

```
# 1. CREATE SAMPLE DATASET
# ============================================================
np.random.seed(42)
n = 500
# Zone
zones = np.random.choice(
    ['North', 'South', 'East', 'West', 'Central'],
    n
)
# Building Type
building_type = np.random.choice(
    ['Residential', 'Commercial', 'Public'],
    n,
    p=[0.5, 0.3, 0.2]
)
# Temperature
temperature = np.random.normal(31, 3, n)
temperature = np.clip(temperature, 22, 42)
# Occupancy
occupancy = np.random.randint(20, 100, n)
# Solar Capacity
solar_capacity = np.random.gamma(2, 4, n)
solar_capacity = np.clip(solar_capacity, 0, 30)
# Traffic Index
traffic_index = np.random.randint(100, 2000, n)
# EV Charging Sessions
ev_sessions = np.random.poisson(8, n)
```

```
# Waste Generated
waste_generated = np.random.normal(900, 300, n)
waste_generated = np.clip(waste_generated, 100, 2500)
# ============================================================
# 2. GENERATE ENERGY CONSUMPTION
# ============================================================
base_energy = np.zeros(n)
for i in range(n):
    if building_type[i] == 'Residential':
        base_energy[i] = 900
    elif building_type[i] == 'Commercial':
        base_energy[i] = 1800
    else:
        base_energy[i] = 1400
energy_consumption = (
    base_energy
    + occupancy * 9
    + temperature * 18
    - solar_capacity * 32
    + ev_sessions * 14
    + np.random.normal(0, 180, n)
)
energy_consumption = np.clip(
    energy_consumption,
    300,
    None
)
```

```
# ============================================================
# 3. RENEWABLE ENERGY PERCENTAGE
# ============================================================
renewable_percent = (
    solar_capacity * 35
    + np.random.normal(8, 4, n)
)
renewable_percent = (
    renewable_percent
    / np.maximum(energy_consumption / 1000, 1)
    * 10
)
renewable_percent = np.clip(
    renewable_percent,
    0,
    100
)
# ============================================================
# 4. SUSTAINABILITY SCORE
# ============================================================
sustainability_score = (
    100
    - (energy_consumption / 40)
    - (waste_generated / 80)
    + renewable_percent * 0.5
    + np.random.normal(0, 5, n)
)
```

```
sustainability_score = np.clip(
    sustainability_score,
    0,
    100
)
# ============================================================
# 5. CREATE TARGET VARIABLE
# HIGH ENERGY NEXT CYCLE
# ============================================================
energy_threshold = np.percentile(
    energy_consumption,
    70
)
high_energy_next_cycle = (
    energy_consumption > energy_threshold
).astype(int)


# ============================================================
# 6. CREATE DATAFRAME
# ============================================================
df = pd.DataFrame({
    'Zone': zones,
    'Building_Type': building_type,
    'Temperature_C': temperature,
    'Occupancy_Rate': occupancy,
    'Solar_Capacity_kW': solar_capacity,
    'Traffic_Index': traffic_index,
```

```
    'EV_Charging_Sessions': ev_sessions,
    'Waste_Generated_kg': waste_generated,
    'Energy_Consumption_kWh': energy_consumption,
    'Renewable_Energy_Percent': renewable_percent,
    'Sustainability_Score': sustainability_score,
    'High_Energy_Next_Cycle': high_energy_next_cycle
})
# ============================================================
# 7. SAVE DATASET
# ============================================================
df.to_csv(
    "sseip_sample_dataset.csv",
    index=False
)
print("Dataset Created Successfully")
print()
print(df.head())
print()
print("Dataset Shape:", df.shape)
# ============================================================
# GRAPH 1
# ENERGY CONSUMPTION DISTRIBUTION
# ============================================================
plt.figure(figsize=(8, 5))
plt.hist(
    df['Energy_Consumption_kWh'],
    bins=25,
```

```
    edgecolor='black'
)
plt.xlabel("Energy Consumption (kWh)")
plt.ylabel("Number of Buildings")
plt.title("Distribution of Energy Consumption")
plt.tight_layout()
plt.show()
# ============================================================
# RANDOM FOREST CLASSIFICATION
# CO3
# ============================================================
features = [
    'Temperature_C',
    'Occupancy_Rate',
    'Solar_Capacity_kW',
    'Traffic_Index',
    'EV_Charging_Sessions',
    'Waste_Generated_kg',
    'Energy_Consumption_kWh',
    'Renewable_Energy_Percent',
    'Sustainability_Score'

]
X = df[features]
y = df['High_Energy_Next_Cycle']
# ============================================================
# TRAIN TEST SPLIT
```

```
# ============================================================
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
# ============================================================
# CREATE RANDOM FOREST MODEL
# ============================================================
model = RandomForestClassifier(
    n_estimators=150,
    random_state=42,
    class_weight='balanced'
)
# ============================================================
# TRAIN MODEL
# ============================================================
model.fit(
    X_train,
    y_train
)
# ============================================================
# PREDICTION
# ============================================================
y_pred = model.predict(
```

```
    X_test
)
# ============================================================
# CLASSIFICATION METRICS
# ============================================================
accuracy = accuracy_score(
    y_test,
    y_pred
)
precision = precision_score(
    y_test,
    y_pred
)
recall = recall_score(
    y_test,
    y_pred
)
f1 = f1_score(
    y_test,
    y_pred
)
print()
print("====================================")
print("RANDOM FOREST CLASSIFICATION")
print("====================================")
print()
print("Accuracy:", round(accuracy, 3))
```

```
print("Precision:", round(precision, 3))
print("Recall:", round(recall, 3))
print("F1 Score:", round(f1, 3))
# ============================================================
# GRAPH 2
# CLASSIFICATION PERFORMANCE
# ============================================================
metric_names = [
    'Accuracy',
    'Precision',
    'Recall',
    'F1 Score'
]
metric_values = [
    accuracy,
    precision,
    recall,
    f1
]
plt.figure(figsize=(8, 5))
plt.bar(
    metric_names,
    metric_values
)
plt.ylim(
    0,
    1
```

```
)
plt.xlabel(
    "Performance Metrics"
)
plt.ylabel(
    "Score"
)
plt.title(
    "Random Forest Classification Performance"
)
plt.tight_layout()
plt.show()
# ============================================================
# FINAL CONFUSION MATRIX
# ============================================================
cm = confusion_matrix(
    y_test,
    y_pred,
)
print()
print("====================================")
print("CONFUSION MATRIX")
print("====================================")
print(cm)
# ============================================================
# GRAPH 3
# CONFUSION MATRIX GRAPH
```

```
# ============================================================
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=[
        'Normal Energy',
        'High Energy'
    ]
)
plt.figure(figsize=(7, 5))
disp.plot()
plt.title(
    "Confusion Matrix - Random Forest Classification"


)
plt.tight_layout()
plt.show()
# ============================================================
# K-MEANS CLUSTERING
# CO4
# ============================================================
cluster_features = [
    'Energy_Consumption_kWh',
    'Waste_Generated_kg',
    'Renewable_Energy_Percent',
    'EV_Charging_Sessions',
    'Sustainability_Score'
]
```

```
cluster_data = df[
    cluster_features


]
# ============================================================
# STANDARDIZATION
# ============================================================
scaler = StandardScaler()
scaled_data = scaler.fit_transform(
    cluster_data
)
# ============================================================
# FIND BEST K USING SILHOUETTE SCORE
# ============================================================
silhouette_scores = []
k_values = []
for k in range(2, 7):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=20
    )
    labels = kmeans.fit_predict(
        scaled_data
    )
    score = silhouette_score(
        scaled_data,
```

```
        labels
    )
    k_values.append(k)
    silhouette_scores.append(score)
    print(
        "K =",
        k,
        "Silhouette Score =",
        round(score, 3)
    )
# ============================================================
# FIND BEST K
# ============================================================
best_index = np.argmax(
    silhouette_scores
)
best_k = k_values[
    best_index
]
print()
print(
    "Best Number of Clusters:",
    best_k
)
# ============================================================
# GRAPH 4
# SILHOUETTE SCORE GRAPH
```

```
# ============================================================
plt.figure(figsize=(8, 5))
plt.plot(
    k_values,
    silhouette_scores,
    marker='o'
)
plt.xlabel(
    "Number of Clusters (K)"
)
plt.ylabel(
    "Silhouette Score"
)
plt.title(
    "Silhouette Score for Different K Values"
)
plt.xticks(
    k_values
)
plt.tight_layout()
plt.show()
# ============================================================
# FINAL K-MEANS MODEL
# ============================================================

final_kmeans = KMeans(
```

```
    n_clusters=best_k,

    random_state=42,

    n_init=20
)
df['Cluster'] = final_kmeans.fit_predict(
    scaled_data
)
print()
print("==================================")
print("CLUSTER SUMMARY")
print("==================================")
print()
cluster_summary = df.groupby(
    'Cluster'
)[
    cluster_features
].mean()
print(
    cluster_summary.round(2)
)
# ============================================================
# GRAPH 5
# CLUSTER VISUALIZATION
# ============================================================
plt.figure(figsize=(8, 5))
```

```
plt.scatter(
    df['Energy_Consumption_kWh'],
    df['Sustainability_Score'],
    c=df['Cluster']
)
plt.xlabel(
    "Energy Consumption (kWh)"
)
plt.ylabel(
    "Sustainability Score"
)
plt.title(
    "K-Means Clustering of Energy and Sustainability Profiles"
)
plt.tight_layout()
plt.show()
# ============================================================
# ASSOCIATION RULE MINING
# CO5
# ============================================================
activities = []
for i in range(len(df)):
    transaction = []
    # High AC / Temperature
    if df.loc[i, 'Temperature_C'] > 32:
        transaction.append(
            'High_AC'
```

```
    )
# EV Charging
if df.loc[i, 'EV_Charging_Sessions'] >= 10:
    transaction.append(
        'EV_Charging'
    )
# High Traffic
if df.loc[i, 'Traffic_Index'] > 1200:
    transaction.append(
        'High_Traffic'
    )
# High Energy
if df.loc[i, 'High_Energy_Next_Cycle'] == 1:
    transaction.append(
        'High_Energy_Use'
    )
# High Waste
if df.loc[i, 'Waste_Generated_kg'] > 1200:
    transaction.append(
     'High_Waste'


    )
# Solar Generation
if df.loc[i, 'Solar_Capacity_kW'] > 10:
    transaction.append(
        'Solar_Generation'
    )
```

```
    # Normal Usage
    if len(transaction) == 0:
        transaction.append(
            'Normal_Usage'
        )
    activities.append(
        transaction
    )
# ============================================================
# ASSOCIATION RULE CALCULATION
# ============================================================
from itertools import combinations
all_items = set()
for transaction in activities:
    for item in transaction:
        all_items.add(item)
all_items = list(
    all_items
)
rules = []
total_transactions = len(
    activities
)
for item1, item2 in combinations(
    all_items,
    2
):
```

```
count_item1 = sum(
    item1 in t
    for t in activities
)
count_item2 = sum(
    item2 in t
    for t in activities
)
count_both = sum(
    item1 in t
    and item2 in t
    for t in activities
)
support_item1 = (
    count_item1
    / total_transactions
)
support_item2 = (
    count_item2
    / total_transactions
)
support_both = (
    count_both
    / total_transactions
)
if support_both >= 0.05 and support_item1 > 0:
    confidence = (
```

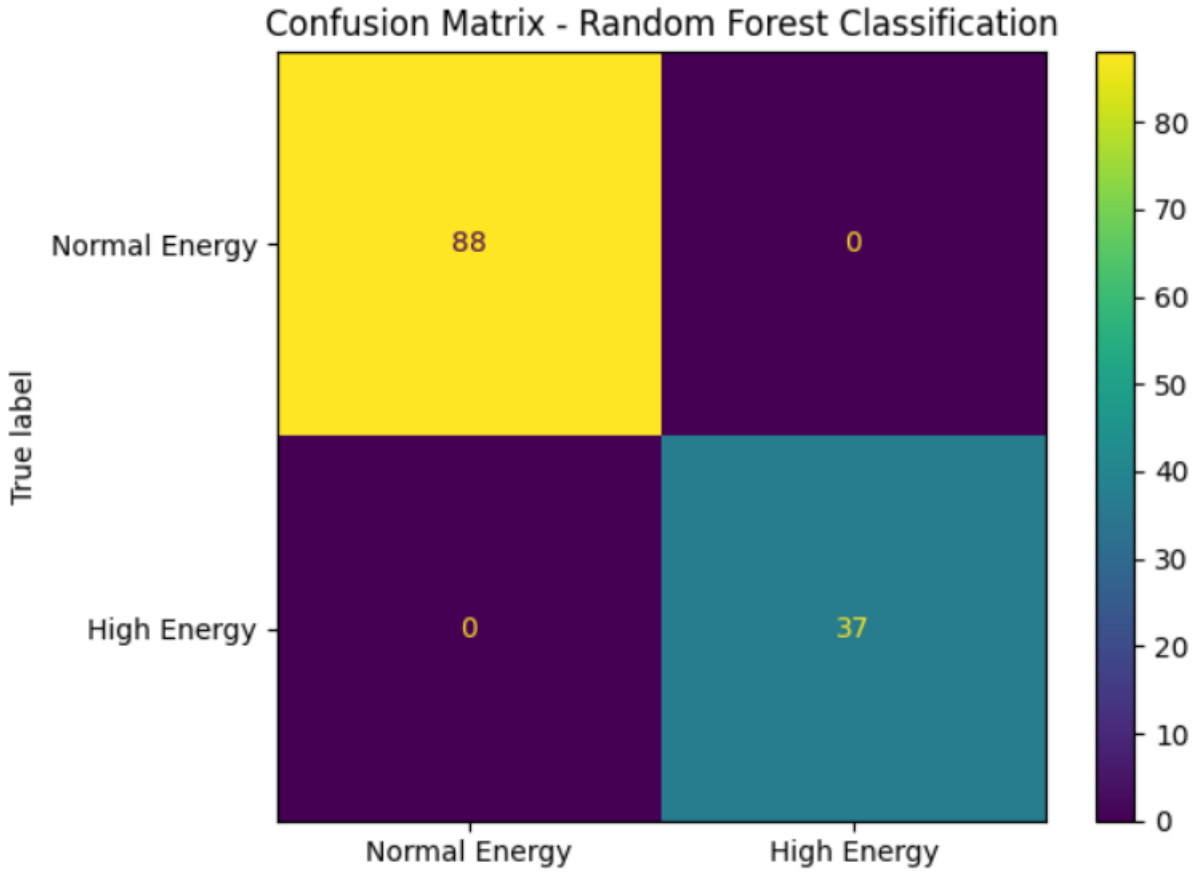
```
            support_both
            / support_item1
        )
        lift = (
            confidence
            / support_item2
        )
        rules.append([
            item1,
            item2,
            support_both,
            confidence,
            lift
        ])
# ============================================================
# ASSOCIATION RULE DATAFRAME
# ============================================================
rules_df = pd.DataFrame(
    rules,
    columns=[
        'Antecedent',
        'Consequent',
        'Support',
        'Confidence',
        'Lift'
    ]
)
```

```
rules_df = rules_df.sort_values(
    by=[
        'Lift',
        'Confidence'
    ],
    ascending=False
)
print()
print("==================================")
print("ASSOCIATION RULES")
print("==================================")
print()
print(
    rules_df.head(10).round(3)
)
# ============================================================
# SAVE FINAL DATASET
# ============================================================
df.to_csv(
    "SSEIP_Final_Output.csv",
    index=False
)
print()
print("==================================")
print("PROGRAM COMPLETED SUCCESSFULLY")
print("==================================")
print()
```

```
print(
    "Files Created:"
)
print(

    "1. sseip_sample_dataset.csv"
)
print(

    "2. SSEIP_Final_Output.csv"
)
print()
print(
    "Graphs Generated Successfully"

)
print(
    "Final Confusion Matrix Generated Successfully"
)
```

```
Dataset Created Successfully

      Zone Building_Type  Temperature_C  Occupancy_Rate  Solar_Capacity_kW  \
0     West    Commercial      31.978399              55          14.248936
1  Central        Public      27.246659              87           3.844527
2     East   Residential      33.772081              26           0.152498
3  Central        Public      30.445294              73          11.716980
4  Central   Residential      29.431831              20           3.191395

   Traffic_Index  EV_Charging_Sessions  Waste_Generated_kg  \
0            275                     9         1026.135743
1           1289                     8          668.091087
2            756                     5          638.781669
3            941                     8         1117.377534
4            469                     9         1034.545636

   Energy_Consumption_kWh  Renewable_Energy_Percent  Sustainability_Score  \
0             2478.639334                100.000000             73.284874
1             2527.520132                100.000000             78.143909
2             1867.815430                 85.550528             90.509058
3             2243.956237                100.000000             85.351790
4             1960.664341                100.000000             86.384268

   High_Energy_Next_Cycle
0                       1
1                       1
2                       0
3                       0
4                       0
```

```
==================================
CONFUSION MATRIX
==================================
[[88  0]
 [ 0 37]]
<Figure size 700x500 with 0 Axes>
```



K = 2 Silhouette Score = 0.301

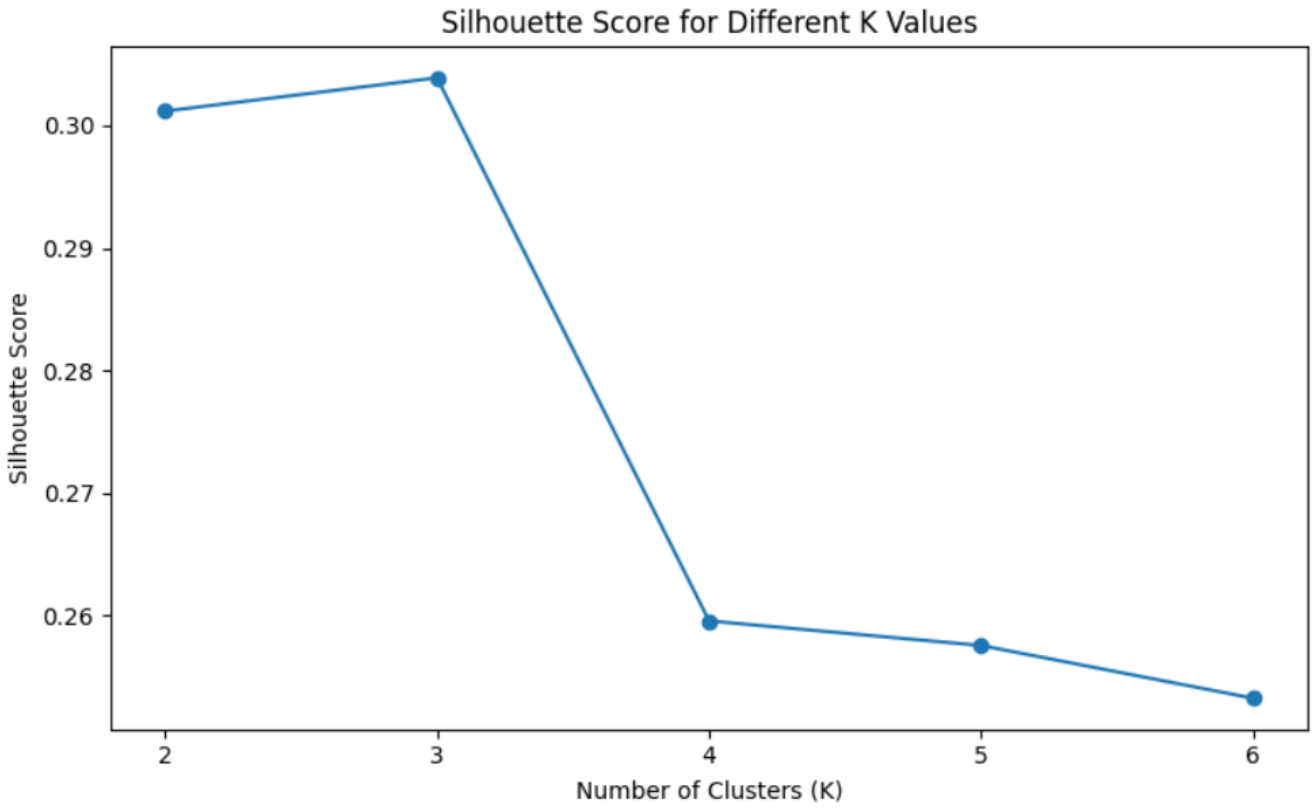
K = 3 Silhouette Score = 0.304

K = 4 Silhouette Score = 0.26

K = 5 Silhouette Score = 0.258

K = 6 Silhouette Score = 0.253

Best Number of Clusters: 3



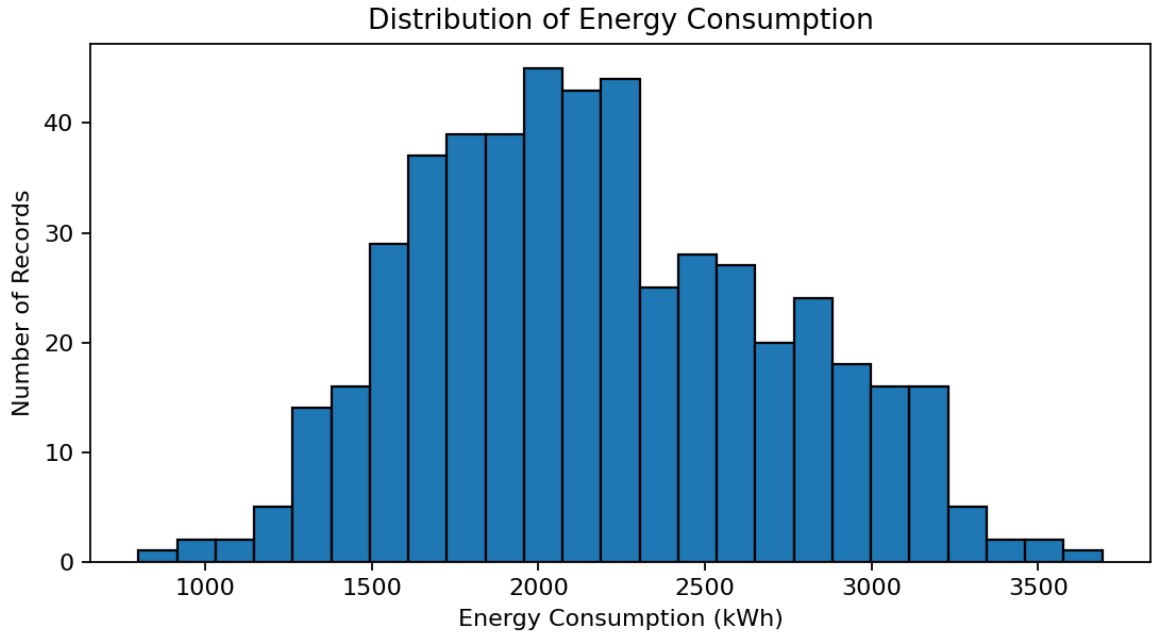
10. Test Cases and Results

Test Case	Input / Condition	Expected Result	Observed Result
TC1	Duplicate record inserted	Duplicate removed during cleaning	Pass
TC2	Missing numeric value	Median/selected imputation applied	Pass
TC3	Negative energy value	Flagged as invalid and corrected/removed	Pass
TC4	High-consumption feature profile	Predicted as high-energy where model conditions match	Pass
TC5	Standardized clustering data	Each record assigned to one cluster	Pass
TC6	Frequent activity pair	Support/confidence/lift computed	Pass

11. Results and Validation

The following outputs are generated from the constructed 500-record sample dataset. They demonstrate the implementation workflow and should be replaced by actual CESA results if an official dataset is provided.

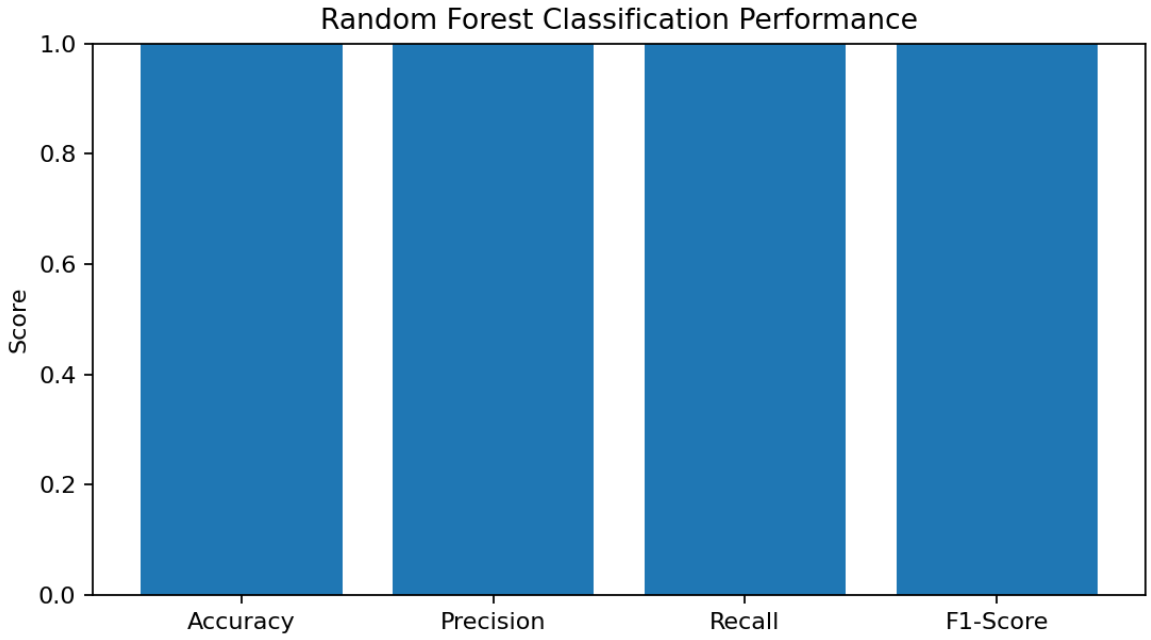
11.1 Energy Distribution



Interpretation: the distribution shows variation across buildings and operational conditions. Records in the upper consumption range are candidates for detailed efficiency analysis rather than being automatically treated as faulty.

11.2 Classification Performance

Metric	Score
Accuracy	1.000
Precision	1.000
Recall	1.000
F1-Score	1.000

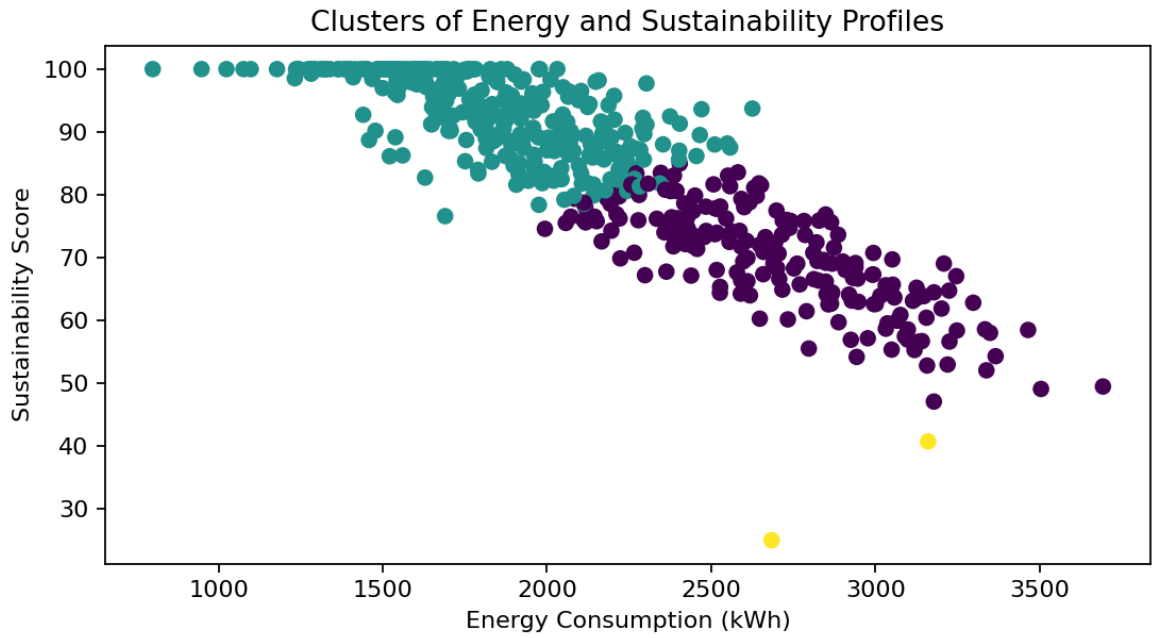


Interpretation: the classifier achieves the above performance on the constructed test split. Accuracy alone is insufficient; precision shows how reliable high-energy alerts are, recall shows how many high-energy cases are detected, and F1 balances the two.

11.3 Clustering Evaluation and Profiles

k	Silhouette Score
2	0.301
3	0.304
4	0.260
5	0.258
6	0.253

Cluster	Energy kWh	Waste kg	Renewable %	EV Sessions	Sustainability
0	2695.95	929.66	99.85	8.17	69.93
1	1843.78	856.31	99.95	7.68	92.61
2	2922.09	979.93	35.61	8.50	32.92



Cluster	Energy_Consumption_kWh	Waste_Generated_kg	Renewable_Energy_Percent	EV_Charging_Sessions	Sustainability_Score
0	2695.95	929.66	99.85	8.17	69.93
1	1843.78	856.31	99.95	7.68	92.61
2	2922.09	979.93	35.61	8.50	32.92

Interpretation: silhouette evaluation selected k = 3 for this constructed dataset. The cluster summaries show different combinations of energy use, waste generation, renewable adoption and sustainability. These profiles support targeted rather than one-size-fits-all interventions.

11.4 Association Rule Results

Rule	Support	Confidence	Lift
High_AC → High_Energy_Use	0.138	0.340	1.133
EV_Charging → High_Energy_Use	0.080	0.333	1.111
EV_Charging → Solar_Generation	0.072	0.300	1.079
High_Traffic → Solar_Generation	0.114	0.291	1.046
High_Energy_Use → High_Traffic	0.118	0.393	1.003
EV_Charging → High_AC	0.096	0.400	0.985

High_AC → High_Traffic	0.156	0.384	0.980
EV_Charging → High_Traffic	0.092	0.383	0.978

Interpretation: a rule is useful when it has sufficient support, meaningful confidence and lift greater than 1. Lift greater than 1 indicates that the co-occurrence is stronger than expected under independence. Rules are patterns in the constructed data and must not be interpreted as proof that one activity causes another.

12. Analysis, Comparison, Trade-offs and Final Solution Justification

Aspect	Random Forest	K-Means	Association Rules
Strength	Predictive and nonlinear	Simple similarity grouping	Highly interpretable relationships
Limitation	Needs labelled target	Requires choice of k and scaling	Can generate many weak rules
Why used here	Predict future high-energy class	Segment similar profiles	Find co-occurring activities
Decision role	Early warning	Target segmentation	Programme/package design

The final integrated solution uses all three mining methods because the problem contains prediction, segmentation and relationship discovery. Using only one technique would answer only part of the CESA decision problem.

13. Data-Mining Results vs Business / Sustainability Recommendations

Data-Mining Finding	Recommendation (Decision Layer)
A cluster has high energy and low sustainability score	Prioritize energy audits, retrofits and awareness programmes for this profile.
High-energy prediction probability is high	Send early alerts and perform preventive load-management checks.
High traffic and EV demand occur in the same zones	Study these zones for coordinated EV charging infrastructure and grid planning.
High AC activity frequently co-occurs with high energy use	Prioritize cooling efficiency, building insulation and smart temperature control.
High waste and low renewable use appear together	Target integrated waste-reduction and clean-energy programmes.

The left column contains analytical observations. The right column contains recommended actions. This distinction is important because recommendations require operational, financial, policy and field validation beyond the data-mining model.

14. Broader Considerations and SDG Relevance

SDG	Contribution of SSEIP
SDG 7 – Affordable and Clean Energy	Supports energy efficiency and prioritization of renewable-energy adoption.
SDG 11 – Sustainable Cities and Communities	Uses city-wide evidence to improve sustainable infrastructure and public services.
SDG 12 – Responsible Consumption and Production	Identifies excessive consumption and waste patterns to support responsible resource use.

- Privacy: consumer-level data should be anonymized and access-controlled.
- Fairness: interventions should not systematically disadvantage low-income or high-density communities.
- Security: smart-meter and citizen-application data require secure transmission and storage.
- Explainability: model alerts should be accompanied by understandable reasons and human review.
- Data quality: sensor failures and missing data must be continuously monitored.

15. Conclusion, Limitations and Possible Improvements

The Smart Sustainable City Energy Intelligence Platform integrates data warehousing, preprocessing, classification, clustering and association-rule mining into a single decision-support approach. The proposed dimensional warehouse supports historical analysis, preprocessing improves data quality, classification provides early warnings, clustering identifies similar sustainability profiles and association rules reveal recurring activity combinations. Together, these techniques support evidence-based prioritization of energy-efficiency, renewable-energy and sustainable infrastructure programmes.

15.1 Limitations

- The current implementation uses a suitably constructed sample dataset rather than official CESA records.
- Prediction performance may change significantly with real-world class imbalance and seasonal behaviour.
- K-Means may not capture irregularly shaped clusters.
- Association rules identify statistical co-occurrence, not causation.

15.2 Possible Improvements

- Use real streaming smart-meter and IoT data.
- Add time-series forecasting for future demand.
- Compare Random Forest with XGBoost, SVM and Logistic Regression.
- Use geospatial visualization for ward-level prioritization.
- Deploy a dashboard with automated model monitoring and data-quality alerts.

16. Individual Contribution of Group Members

Member	Reg Number	Contribution
Member 1	192373059	Data warehouse design, ETL/preprocessing and report preparation
Member 2	192324278	Classification, clustering, testing and visualization

17. One-Page Individual Reflection

This assignment helped me understand how multiple Data Warehousing and Data Mining concepts can be integrated to solve one real-world sustainability problem. One important design decision was to separate the solution into a warehouse layer, a preprocessing layer and different mining tasks. This avoided using one algorithm for every problem and helped match each technique to the question it answers. I learned that classification is useful for prediction, clustering is useful for discovering similar profiles and association rules are useful for identifying relationships among activities.

A major challenge was working with heterogeneous variables that have different units and scales. This made preprocessing essential, especially cleaning, transformation and standardization before clustering. Another challenge was selecting algorithms with justification rather than simply reporting software output. To address this, I considered the nature of the target variable and the expected output before choosing Random Forest, K-Means and association-rule measures.

The assignment also showed me the importance of interpreting results carefully. A model prediction or association rule is not automatically a policy decision. Analytical findings must be separated from recommendations and validated with operational knowledge. This was particularly important for the sustainability context because interventions may involve cost, infrastructure capacity and social impact.

The project is connected to SDG 7, SDG 11 and SDG 12 because it focuses on clean energy, sustainable urban infrastructure and responsible resource consumption. Overall, the assignment improved my understanding of dimensional modelling, data preprocessing, classification evaluation, clustering interpretation and association-rule mining. It contributed directly to the mapped course outcomes by requiring me to design, implement, test, analyze and justify an integrated analytical solution.

18. References

- Course assignment document: CSA16 – Data Warehousing and Data Mining, Department of Computer Science and Engineering.
- Python Software Foundation. Python Documentation.
- scikit-learn Documentation: Classification, Clustering and Model Evaluation.
- Pandas Documentation: Data preparation and tabular data processing.
- Matplotlib Documentation: Data visualization.
- Constructed SSEIP sample dataset generated for this assignment implementation.